CodeSee

An Interactive AI-Powered Tool for Understanding and Fixing Code Errors

Made with GAMMA

What is CodeSee?

CodeSee is an interactive AI-powered tool designed primarily to assist programmers, especially beginners, in understanding and fixing code errors. The platform provides an intuitive interface that transforms the debugging experience from a frustrating struggle into an educational journey.

The core philosophy behind CodeSee is simple yet powerful: rather than just fixing errors, the tool teaches users the logical thinking process to identify and resolve issues independently in the future. This approach bridges the gap between simply getting code to work and truly understanding why it wasn't working in the first place.

Made with GAMMA

Two Modes of Operation

The CodeSee interface presents users with two main buttons, each representing a different approach to debugging and learning:

User Solves Mode

In this educational mode, the system does not provide the direct solution. Instead, it offers a series of intelligent hints and guiding questions designed to help the user pinpoint the error's location, understand its cause, and ultimately arrive at the solution themselves.

The goal here is learning, not just getting the answer. This mode simulates having a patient tutor who guides you through the problem-solving process step by step.

AI Solves Mode

Here, the Agent performs a complete code analysis, identifies the error, and suggests a fix (or multiple potential fixes) along with a simple, clear explanation of why the error occurred and how to resolve it.

This mode is perfect when you need a quick solution or want to compare your own approach with the AI's recommended fix. It provides immediate feedback and detailed explanations.

Made with GAMMA

Agent Types: The Technical Foundation

CodeSee employs four distinct types of AI agents, each with specific capabilities and responsibilities:

01

Simple Reflex Agent

Technology: If-Then Rules

Responsibility: Immediate and direct responses to well-known patterns

Example: If the user inputs code with a very simple syntax error (like forgetting a colon after a function definition), this Agent recognizes the pattern immediately and gives an instant alert.

Rule: (If error == known syntax error) → (Display quick warning)

02

Model-based Agent

Technology: CSP + AST Parsing

Responsibility: Provide accurate hints in "User Solves" mode, taking into account conversation history. In "AI Solves" mode, suggests fixes that respect the entire code structure.

Function: Building an internal model of the code, including variable relationships and logical constraints, and using this model to provide accurate, context-aware hints.

03

Goal-based Agent

Technology: BFS / DFS / A*

Responsibility: Primarily activated in "User Solves" mode. Its goal is not just to give information, but for the user to reach the correct solution by themselves.

Function: Every hint or question it provides is a calculated step towards achieving this goal. It searches the space of possible actions (different hints) and chooses the action it believes will bring the user closer to their final goal.

04

Utility-based Agent

Technology: A* + Genetic Algorithm

Responsibility: Primarily activated in "AI Solves" mode, and can also be used to evaluate hints in the other mode. When there is more than one possible solution to an error.

Function: Evaluating multiple solutions and choosing the best based on a Utility Function that considers criteria like simplicity, efficiency, and safety.

Made with GAMMA

Why CodeSee? The Problem We're Solving

The Time Drain

Programmers, especially beginners, spend a significant amount of time debugging, sometimes for simple errors that could be resolved quickly with proper guidance.

The Learning Gap

Existing tools either provide the solution directly (which doesn't promote deep learning) or merely point out the error without meaningful interaction or educational context.

The Need for Interaction

There's a clear gap in tools that can act as a "private tutor," guiding users step-by-step through the debugging process while building their problem-solving skills.

Made with GAMMA

Our Goal

The Mission

To create an interactive educational tool that doesn't just fix errors, but teaches the user the logical thinking process to identify and fix errors independently in the future.

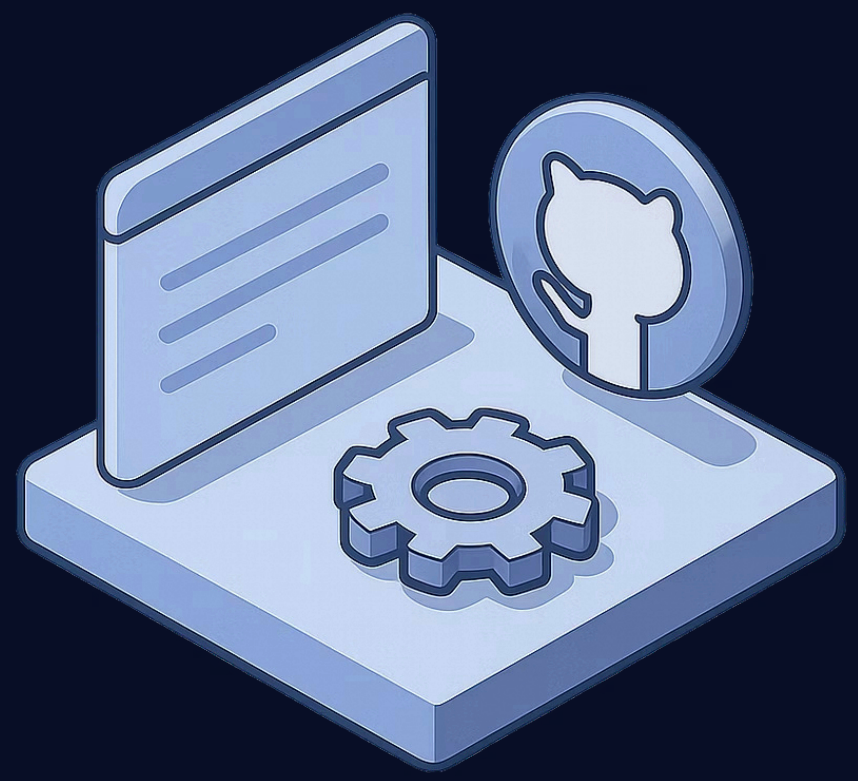
CodeSee aims to transform debugging from a source of frustration into a valuable learning opportunity. By providing guided hints and explanations, the tool helps users build the mental models and problem-solving strategies that expert programmers use naturally.

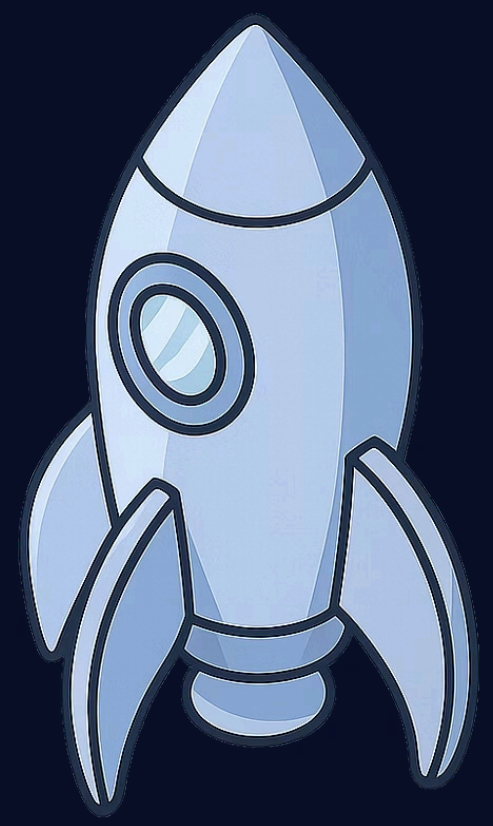
The ultimate goal is to reduce dependency on external help while increasing confidence and competence in debugging. Users who regularly use CodeSee should find themselves spending less time stuck on errors and more time building features.

Made with GAMMA

Related Work: Existing Solutions

While CodeSee offers a unique interactive approach, it's important to understand the landscape of existing tools:





GitHub Copilot

An AI pair programmer that suggests entire lines or blocks of code as you type. Users can provide code with an error, and Copilot will instantly suggest a fix. However, it primarily operates on the "AI Solves" logic and is not specifically designed to act as a guiding tutor.

ChatGPT

A general-purpose AI chatbot that can analyze code and suggest fixes when provided with error messages or problematic code. Like Copilot, it excels at providing solutions but doesn't offer a structured, step-by-step learning path through the debugging process.

These are the most prominent examples in the space, but they lack the educational scaffolding that makes CodeSee unique. They're excellent tools for getting code working quickly, but they don't necessarily teach the underlying problem-solving skills.

Made with GAMMA

PEAS Framework: Performance Elements

PEAS is a framework used to define the characteristics of intelligent agents. Let's break down how CodeSee fits into each category:

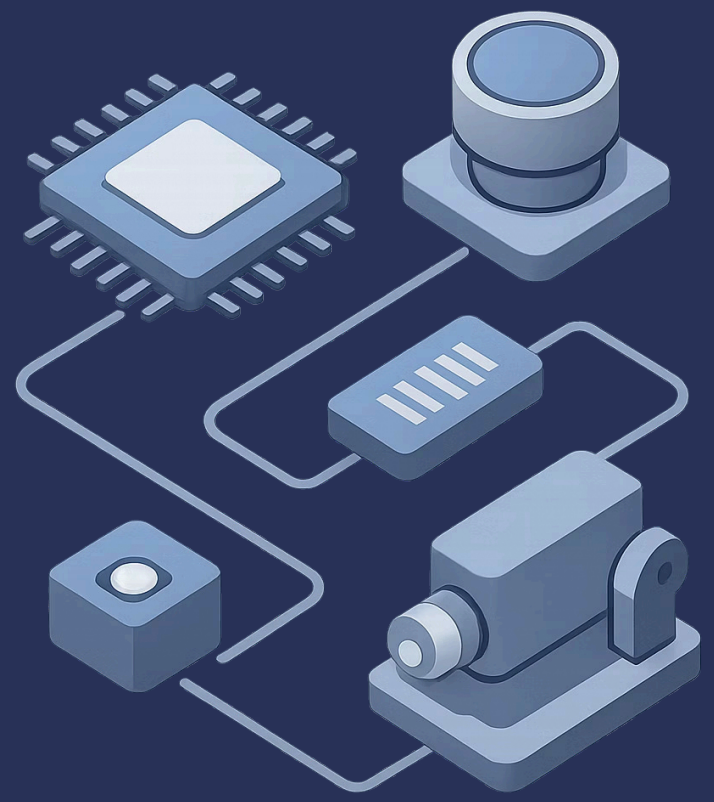
Performance Measure

- **Accuracy** in identifying the error
- **Quality** of the suggested solution
- **Effectiveness** of hints in helping the user
- **Response speed** and system performance

Environment

- The **Web Application platform**
- The **source code** input by the user
- The **history of past interactions** with the user
- The environment is the code itself, the errors present in it, and any modifications the user makes

Made with GAMMA



PEAS Framework: Actuators & Sensors

Actuators

Actuators are how the agent affects its environment:

- Displaying hints on the screen
- Displaying suggested solutions
- Displaying guiding questions
- Highlighting the error line
- Suggesting similar practice exercises

The system can:

- Add `;`
- Fix braces `{}`
- Identify an undefined variable
- Add return statements
- Delete or modify a wrong line

Sensors

Sensors are how the agent perceives its environment:

- Reading the input code
- Detecting button clicks
- Reading error messages from helper tools
- Recording user ratings of suggestions
- Tracking the time taken by the user to solve

Made with GAMMA

ODESA Framework: Environment Properties

ODESA describes the fundamental properties of the environment in which the agent operates:

Observability: Partial

The Agent sees the code and interaction history, but not the user's true intent, experience level, or full project context.

Deterministic: Stochastic

Outcomes are not 100% guaranteed. The same hint might affect different users differently. The success of the learning process depends on uncertain factors.

Episodic: Sequential

Current decisions depend heavily on past interactions. The Learning agent learns from previous episodes to improve performance in future episodes.

Static: Dynamic

The environment (code, user input, and even the Agent's knowledge) changes constantly during a debugging session and in the long term with learning.

Agents: Multi-agent

There is more than one influencing entity: the AI program (which itself is a collection of cooperating Agents) and the user. The internal components of the Learning agent can be seen as cooperating sub-agents.

Made with GAMMA